



Card Cafe

เอกสารอธิบายโค้ด (Technical Deep-Dive) — สำหรับอ่านทบทวนและเตรียมอธิบายทุกส่วนของระบบ

โปรเจกต์: Card Cafe — Walk-around virtual card shop

Stack: Phaser 3 + Vite (client) · Node.js + Express + Socket.io + PostgreSQL (server)

จุดประสงค์เอกสาร: อธิบายสถาปัตยกรรม การตัดสินใจเชิงออกแบบ และโค้ดแต่ละส่วน ให้พร้อมอธิบายได้เองทุกจุด (เช่น ตอนสัมภาษณ์งาน)

สารบัญ

- ภาพรวมโปรเจกต์ & Tech Stack
- สถาปัตยกรรมระดับสูง & โครงสร้างโฟลเดอร์
- ระบบบัญชีผู้ใช้ & การยืนยันตัวตน (Auth)
- แกนกลาง Realtime (Socket.io & สถานะในหน่วยความจำ)
- ฝั่ง Client — Phaser Scene & การเคลื่อนที่
- ระบบแชท (Shop-wide / Table-scoped)
- ระบบ Voice & Video Call (WebRTC Mesh)
- ระบบ Shop-Owner & Marketplace
- ฐานข้อมูล & Migration Workflow
- การตัดสินใจเชิงสถาปัตยกรรม & Trade-offs สำคัญ
- คำถามที่มักถูกถามตอนสัมภาษณ์ + แนวทางตอบ
- Roadmap / สิ่งที่ยังไม่ได้ทำ

1. ภาพรวมโปรเจกต์ & Tech Stack

Card Cafe คือเว็บแอปแนว "walk-around virtual space" (คล้าย Gather.town) สำหรับเล่นเกมการ์ดออนไลน์ร่วมกัน ผู้เล่นควบคุมตัวละคร 2D เดินไปมาในร้าน เดินไปนั่งที่โต๊ะ แล้วแชท/วิดีโอคอลกับคนที่นั่งโต๊ะเดียวกันเท่านั้น (ไม่ใช่แชทรวมทั้งห้องตลอดเวลาแบบ Zoom) — เป็นจุดขายหลักของ UX แบบนี้

Elevator pitch (พูดสั้นๆ ตอนถูกถามว่า "โปรเจกต์นี้ทำอะไร")

"มันคือพื้นที่เสมือนที่เดินไปมาได้ พอเดินไปนั่งโต๊ะไหน ก็จะเข้ากลุ่มแชท/วอยซ์คอลของโต๊ะนั้นอัตโนมัติ ไม่ต้องกดปุ่ม join ห้อง มีระบบบัญชีผู้ใช้ มีร้านหลายร้านแยกกันด้วยรหัสร้าน และมีพีเจอาร์ให้คนเปิดร้านขายของตัวเองได้ โดยต้องผ่านแอดมินตรวจสอบก่อน"

Tech Stack

ส่วน	เทคโนโลยี	หน้าที่
Client	Phaser 3, Vite, Vanilla JS (ES Modules)	เรนเดอร์เกม 2D, UI panel (แชท/คอล/ตลาด) เป็น HTML overlay แยกจาก canvas
Server	Node.js, Express, Socket.io	HTTP REST (auth, admin, marketplace) + WebSocket (ตำแหน่งผู้เล่น, แชท, สัญญาณ WebRTC)
Database	PostgreSQL (ผ่าน pg driver ตรงๆ ไม่มี ORM)	เก็บบัญชี, ร้าน, ประวัติแชท, ใบสมัครเจ้าของร้าน, สินค้า, ออเดอร์
Auth	JWT (jsonwebtoken) + bcrypt	ออก token อายุ 30 วัน, เก็บรหัสผ่านแบบ hash
Realtime media	WebRTC (mesh, ไม่มี media server)	วิดีโอ/วอยซ์คอลต่อโต๊ะ, สัญญาณ (signaling) ส่งผ่าน Socket.io

โปรเจกต์แบ่งเป็น 2 โฟลเดอร์หลัก `client/` และ `server/` เป็นคนละ npm package กัน ตั้งใจให้แยก deploy ได้ในอนาคต (client ขึ้น static host, server ขึ้น Node host เล็กๆ)

2. สถาปัตยกรรมระดับสูง & โครงสร้างโฟลเดอร์

หลักการสำคัญที่สุด: Server เป็นเจ้าของ state ทั้งหมด

Client **ไม่เคย hardcode** ว่าร้านหน้าตาเป็นยังไง (ขนาดพื้น, ตำแหน่งโต๊ะ) — ทุกอย่างมาจาก server ตอน join (`join-shop` ack) ถ้าจะเพิ่มร้าน/อีเวนต์ใหม่ แค่ฝั่ง server (`server/src/scenes/`) ฝั่ง client ไม่ต้องแตะเลย เพราะมันแค่ "เรนเดอร์สิ่งที่ server บอกมา"

เช่นเดียวกัน client ไม่เคยเห็น connection string ของ Postgres เลย — คุยกับ DB ได้ทาง server เท่านั้น (ผ่าน HTTP REST หรือ Socket.io)

โครงสร้างโฟลเดอร์

```
port1_playground/
├── server/
│   ├── src/
│   │   ├── index.js           # จุดเริ่มต้น: Express app + Socket.io server + socket event handlers ทั้งหมด
│   │   ├── auth.js           # hash/verify password, sign/verify JWT
│   │   ├── rooms.js          # state ผนังแบบ in-memory (ไม่ลง DB)
│   │   ├── middleware/auth.js # requireAuth / requireRole สำหรับ REST route
│   │   ├── scenes/           # เลย์เอาต์ร้าน (ไฟล์ static + template สำหรับเจ้าของร้าน)
│   │   ├── db/               # query ไป Postgres ทั้งหมด (shops.js, marketplace.js, pool.js)
│   │   ├── routes/           # REST endpoint: auth, admin, shops, orders, ownerApplications
│   │   ├── migrations/       # ไฟล์ .sql แบบ hand-written
│   │   │   └── scripts/       # migrate.js, set-role.js
│   └── package.json
└── client/
    ├── src/
    │   ├── main.js           # ผูกทุกอย่างเข้าด้วยกัน (auth form, lobby, เรียก initXxxUI ต่างๆ)
    │   ├── scenes/ShopScene.js # Phaser scene หลัก: เรนเดอร์ + input + physics ง่ายๆ
    │   ├── ui/               # DOM overlay ล้วนๆ: chat.js, call.js, adminPanel.js ฯลฯ
    │   ├── webrtc/callManager.js # WebRTC mesh logic
    │   ├── net/              # socket.js, api.js (fetch wrapper)
    │   └── auth/session.js    # เก็บ token ใน localStorage
    └── package.json
```

จุดสังเกตสำหรับอธิบาย: UI ที่ไม่ใช่ตัวเกม (แชท, ปุ่มคอล, หน้าแอดมิน) ทำเป็น **DOM** ส่วนๆ ทับอยู่บน **canvas** ของ Phaser ไม่ได้วาดด้วย Phaser's text object — เพราะ input field, scroll แชท ฯลฯ ทำง่ายกว่ามากด้วย HTML ปกติ Phaser จัดการแค่ "โลกเกม" (ตัวละคร, โต๊ะ, การเดิน)

3. ระบบบัญชีผู้ใช้ & การยืนยันตัวตน (Auth)

ไม่มี guest/anonymous เล่นได้ — ต้องสมัครสมาชิกก่อนเสมอ (`server/src/routes/auth.js`)

Flow การสมัคร/ล็อกอิน

1. `POST /auth/register` → validate email/password/displayName → `bcrypt.hash()` รหัสผ่าน → insert ลง `users` → ออก JWT (`signToken`)
2. Client เก็บ token ไว้ใน `localStorage` (`client/src/auth/session.js`)
3. ทุก request ต่อไปแนบ token: REST ผ่าน header `Authorization: Bearer <token>` , Socket.io ผ่าน `auth.token` ตอน handshake

Token มีอะไรอยู่ข้างใน

```
// server/src/auth.js
jwt.sign({ sub: user.id, displayName: user.display_name }, secret, { expiresIn: "30d" })
```

สังเกตว่า token เก็บแค่ `userId` กับ `displayName` — ไม่เก็บ `role` ไว้ใน token โดยตั้งใจ (อธิบายต่อด้านล่าง)

requireAuth vs requireRole — ทำไมต้อง query DB ซ้ำทุกครั้ง

จุดนี้สำคัญมาก เป็นการตัดสินใจเชิงออกแบบที่ดีมากสำหรับอธิบายตอนสัมภาษณ์:

```
// server/src/middleware/auth.js
export function requireRole(...allowedRoles) {
  return async (req, res, next) => {
    const { rows } = await pool.query("SELECT role FROM users WHERE id = $1", [req.auth.userId]);
    const role = rows[0]?.role;
    if (!role || !allowedRoles.includes(role)) {
      return res.status(403).json({ error: "..." });
    }
    req.auth.role = role;
    next();
  };
}
```

ทำไมไม่เก็บ role ไว้ใน JWT เลย แล้วเช็คจาก token อย่างเดียว?

เพราะ token มีอายุ 30 วัน ถ้าเก็บ role ไว้ใน token ตอนล็อกอิน แล้วแอดมินอนุมัติให้ user คนนั้นเป็น `shop_owner` ทีหลัง token เก่าที่ user ถืออยู่จะยัง "จำ" ว่าตัวเองเป็น `customer` ไปอีกตั้ง 30 วัน (หรือจนกว่าจะ logout/login ใหม่) การ query role จาก DB สดทุกครั้งทำให้สิทธิ์ใหม่ "มีผลทันทีตั้งแต่ request ถัดไป" ไม่ต้องให้ user ออกจากระบบแล้วเข้าใหม่

Role ที่มีในระบบ

`customer` (ค่าเริ่มต้น) → `shop_owner` (อนุมัติโดยแอดมิน) → `admin` (ไม่มีทางสมัครเอง ต้องตั้งผ่าน CLI script `scripts/set-role.js` เท่านั้น)

4. แกนกลาง Realtime (Socket.io & State ในหน่วยความจำ)

Socket handshake ต้องผ่าน middleware ก่อนเสมอ

```
// server/src/index.js
io.use((socket, next) => {
  const token = socket.handshake.auth?.token;
  const user = verifyToken(token);
  if (!user) return next(new Error("unauthorized"));
  socket.user = user; // { userId, displayName }
  next();
});
```

ถ้า token ไม่ผ่าน การเชื่อมต่อ socket จะถูกปฏิเสธตั้งแต่ขั้น handshake — ไม่มี socket เปล่าๆ ที่ไม่มีเจ้าของหลุดเข้ามาได้

State 2 ชนิดที่แยกกันเด็ดขาด

	อยู่ที่ไหน	ตัวอย่าง	รอด server restart ไหม
Live presence	<code>server/src/rooms.js</code> (in-memory <code>Map</code>)	ตำแหน่ง x/y, กำลังนั่งโต๊ะไหน, กำลังคอลอยู่ไหม	✗ ไม่ ไม่จำเป็นต้องรอด
Durable data	Postgres (<code>server/src/db/</code>)	บัญชีผู้ใช้, ร้าน, ประวัติแชท, สินค้า, ออเดอร์	✔ รอด

เหตุผล: ตำแหน่งผู้เล่นเปลี่ยนเป็นสิบครั้งต่อวินาที ถ้าเขียนลง Postgres ทุกครั้งจะพังเรื่อง performance และไม่มีใครสนใจตำแหน่งเก่าหลัง server รีสตาร์ทอยู่แล้ว (ทุกคน reconnect ใหม่ สปอว์นใหม่) — ตรงข้ามกับแชท/บัญชี ที่ต้องอยู่ถาวร

Lifecycle ของการเข้าร้าน (event หลักเรียงตามลำดับ)

- join-shop** → หา/สร้างร้านใน Postgres (`findOrCreateShop`) → เช็คว່าร้านเปิดให้คนทั่วไปดูหรือยัง (ถ้าเป็นร้านของเจ้าของร้านที่ยังไม่ publish จะเข้าได้แค่เจ้าของเอง) → สร้าง player ใน `rooms.js` → ส่งกลับ layout + ผู้เล่นคนอื่น + ประวัติแชท 50 ข้อความล่าสุด
- move** → broadcast ตำแหน่งใหม่ไปทุกคนใน shop room (ไม่เขียน DB)
- join-table** (ยิงอัตโนมัติจาก client เมื่อเดินเข้าใกล้โต๊ะ ไม่ใช้กดปุ่ม) → ออกจากห้อง Socket.io ของโต๊ะเก่า (ถ้ามี, และจบคอลที่โต๊ะเก่าด้วย) → เข้าห้องโต๊ะใหม่ → ส่งประวัติแชทของโต๊ะนั้นกลับ
- shop-chat / table-chat** → insert ลง Postgres ก่อน → broadcast ด้วยค่า id/timestamp ที่ได้จาก DB จริง (ไม่ใช่สุ่มฝั่ง client)
- disconnect** → จบคอลถ้ากำลังคอลอยู่ → ลบ player ออกจาก `rooms.js` → บอกทุกคนว่า player นี้ออกไปแล้ว

socketMeta — ตัวเชื่อม socket.id กับ state

Server เก็บ `Map` ชื่อ `socketMeta` (อยู่ใน `index.js` ไม่ใช่ `rooms.js`) แมป `socket.id` → { `roomId, tableId, tables, shopDbId` } ไว้เป็น cache สำหรับ handler อื่นๆ จะได้ไม่ต้อง query ซ้ำว่า "คนนี้อยู่ร้านไหน โต๊ะไหน" ทุกครั้งที่มิ event เข้ามา

5. ผัง Client — Phaser Scene & การเคลื่อนที่

ไฟล์หลักคือ `client/src/scenes/ShopScene.js` — Phaser scene เดียวที่ใช้กับทุกร้าน (เพราะ layout มาจาก server ไม่ใช่ scene ที่ hardcode)

ทำไมการเคลื่อนที่เป็นแบบ "client-authoritative"

Client คำนวณตำแหน่งตัวเองเลย (บวก delta ตาม input ทันที) แล้วค่อยส่ง `move` event ไป server ~20 ครั้ง/วินาที (throttled ด้วย `MOVE_EMIT_INTERVAL = 50ms`) ไม่ใช่รอ server ตอบกลับก่อนขยับ — นี่คือ trade-off ความลื่นไหล (ไม่มี input lag) แลกกับการไม่มี server-side anti-cheat ใดๆ เรื่องตำแหน่ง (เหมาะกับเกมสังสรรค์/การ์ดเกม ไม่ใช่เกมแข่งขันที่ต้อง fair อย่างเข้มงวด)

Remote player interpolation (ทำไมตัวละครคนอื่นดูลื่น ไม่กระตุก)

```
// update() ทุกเฟรม
rp.sprite.x = Phaser.Math.Linear(rp.sprite.x, rp.target.x, REMOTE_LERP); // REMOTE_LERP = 0.25
```

ตำแหน่งที่ได้จาก server (`player-moved`) ไม่ได้เซตตำแหน่งตรงๆ ทันที แต่เก็บไว้เป็น `target` แล้วค่อยๆ เลื่อนเข้าหา (linear interpolation 25% ต่อเฟรม) — วิธีมาตรฐานสำหรับลด "กระตุก" จากการอัปเดตตำแหน่งที่มาไม่ถึงเท่า framerate

Table proximity แทนปุ่ม "Join"

```
updateTableProximity() {
  // หาโต๊ะที่ใกล้ที่สุดที่ยังในรัศมี table.radius
  // ถ้าเปลี่ยนโต๊ะ (รวมถึงเปลี่ยนจาก "ไม่มีโต๊ะ" ↔ "มีโต๊ะ")
  //   → เคลียร์ log แชนโต๊ะเดิม, อัปเดต UI, จบ/เริ่ม callHooks,
  //   ยัง join-table หรือ leave-table ไป server
}
```

ฟังก์ชันนี้ถูกเรียกทุกเฟรมจาก `handleMovement()` — เช็กระยะห่างแบบวงกลม (circle distance) ง่ายๆ ไม่ใช่ physics engine เต็มรูปแบบ เพราะไม่จำเป็นต้องซับซ้อนกว่านี้สำหรับเกมแนวนี้

Collision กับโต๊ะ

ใช้ circle-vs-circle collision ธรรมดา (ไม่ได้ใช้ Phaser physics plugin) — คำนวณระยะห่างจากศูนย์กลางโต๊ะ ถ้าใกล้เกินไปก็ "ดันออก" ตามมุม (angle) แทน ง่ายกว่าและพอเพียงสำหรับโต๊ะทรงกลม/สี่เหลี่ยมมุมมนแบบนี้

6. ระบบแชท (Shop-wide / Table-scoped)

แชทมี 2 ระดับ ใช้ตารางเดียวกัน (`chat_messages`) แยกกันด้วยคอลัมน์ `table_id` :

- **Shop-wide** (`table_id IS NULL`) — ทุกคนในร้านเห็น ไม่ว่าจะยืนตรงไหน
- **Table-scoped** (`table_id` มีค่า) — เห็นเฉพาะคนที่นั่งโต๊ะเดียวกัน ด้วย Socket.io room ชื่อ `${roomId}::${tableId}`

Pattern สำคัญ: Persist ก่อน Broadcast เสมอ

```
const saved = await insertChatMessage({ shopId, tableId, userId, authorName, text });
io.to(room).emit("shop-chat", { id: saved.id, text: saved.text, at: new Date(saved.created_at).getTime() });
```

เขียนลง Postgres ก่อน แล้วค่อย broadcast โดยใช้ `id` / `created_at` ที่ได้จาก DB จริง (ไม่ใช่สุ่มเองที่ client หรือใช้ `Date.now()` ผั่ง server เฉยๆ) เพื่อให้ **ข้อความที่เพิ่ง broadcast ออกไป กับข้อความเดียวกันที่คนเข้ามาทีหลังจะเห็นใน history ตรงกันเป๊ะ** ไม่มีทาง id/เวลาเพี้ยนกันระหว่างสองที่

7. ระบบ Voice & Video Call (WebRTC Mesh)

Mesh คืออะไร ต่างจาก SFU ยังไง

แต่ละคนในคอลเปิด `RTCPeerConnection` ตรงไปหาทุกคนอื่นในคอล (full mesh) — **ไม่มี media server กลาง** server มองไม่เห็นภาพ/เสียงเลย เห็นแค่ signaling message เล็กๆ (offer/answer/ICE candidate) ที่ relay ผ่านไปให้ ข้อดีคือไม่ต้องมี infrastructure เพิ่ม (ไม่ต้องรัน media server แยก) ข้อเสียคือ bandwidth การอัปโหลดของแต่ละคน โตตามจำนวนคนในคอล (ยังเป็นเลขยกกำลังสองสำหรับวิดีโอ เพราะทุกคนต้องส่งให้ทุกคน) — เป็นเหตุผลที่ cap วอยซ์ไว้ 10, วิดีโอแค่ 4

Glare avoidance — กติกา "ใครเข้าใหม่เป็นคนเสนอ offer เสมอ"

```
// callManager.js join()
for (const peer of ack.peers) {      // peer ท้ายในคอลอยุ่แล้ว
  await makeOfferTo(peer.id);        // ผู้เข้าใหม่เป็นผู้ส่ง offer เสมอ
}
// ผู้ส่งคนท้ายในคอลอยุ่แล้ว ไม่เคยส่ง offer เอง มัปเดตตอบ answer
```

ถ้าไม่มีกติกานี้ และทั้งสองฝั่งเสนอ offer พร้อมกัน (glare) จะต้องมี logic "perfect negotiation" ที่ซับซ้อนกว่านี้มาก (rollback description, เทียบ priority) การบังคับว่า "คนเข้าใหม่เท่านั้นที่ offer" ตัดปัญหาที่งงไปเลยตั้งแต่การออกแบบ

Mic mute / Deafen / Speaking indicator (ฟีเจอร์ที่เพิ่งเพิ่ม)

ฟีเจอร์	ทำงานยังไง	broadcast ไปทำไม
Mute mic	<code>track.enabled = false</code> บน audio track ของตัวเอง — ไม่ต้อง renegotiate	ให้คนอื่นเห็น badge 🔇
Deafen	ปิดเสียงที่ได้ยิน โดย mute <code><video></code> element ของทุกคน (ยกเว้นตัวเอง) ฝั่งตัวเองล้วนๆ	ให้คนอื่นเห็น badge 🙊 (คนอื่นไม่ได้ยินเสียงเปลี่ยนอะไรเลย มีผลแค่ฝั่งคนกด)
Deafen → auto-mute	กด deafen จะ mute ไม่ให้ตัดโน้ตด้วย (ถ้ายังไม่ได้ mute เอง) แบบ Discord — จำสถานะด้วยตัวแปร <code>mutedByDeafen</code> เพื่อ undeafen แล้ว unmute กลับให้ถูก	-
Speaking indicator	ใช้ Web Audio API (<code>AnalyserNode</code>) วัดระดับเสียงจาก stream จริง ของทุกคนรวมถึงตัวเอง ทุก 100ms เทียบ threshold	ไม่ broadcast ไปไหน — แต่ละฝั่งวัดเสียงที่ตัวเอง "ได้รับ" จากสตรีมเอง ไม่ต้องพึ่ง server

จุดที่น่าสนใจ: Speaking indicator ไม่ต้องส่งอะไรผ่าน socket เลย เพราะทุกฝั่งมี audio stream จริงอยู่ในมืออยู่แล้ว (จาก WebRTC track) แคเอา Web Audio API ไปปะพัง amplitude เฉยๆ — ประหยัด network เพราะไม่ต้องส่ง "ใครพูดอยู่" เป็น event

Caps ถูกบังคับที่ server เท่านั้น

```
// server/src/rooms.js
export function voiceCountAtTable(roomId, tableId) {
  return listPlayers(roomId).filter((p) => p.tableId === tableId && p.inCall).length;
}
```

`VOICE_LIMIT = 10` , `VIDEO_LIMIT = 4` เช็คจากจำนวนจริงใน `rooms.js` ไม่ใช่เชื่อ client ว่า "โต๊ะนี้เต็มหรือยัง" — client ส่ง flag ผิดยังไงก็เข้าไม่ได้ถ้า server นับแล้วเกิน

ทำไมคอลจบอัตโนมัติเมื่อเดินออกจากโต๊ะ

`endCallForSocket()` ถูกเรียกทั้งตอน `join-table` (เปลี่ยนโต๊ะ), `leave-table` , และ `disconnect` — เพราะคอลถูกนิยามว่า "เฉพาะคนที่กำลังนั่งโต๊ะนี้อยู่ ณ ตอนนี้" ไม่มีทางที่ avatar เดินออกไปแล้วแต่เสียงยังค้างอยู่ในคอลเดิม

8. ระบบ Shop-Owner & Marketplace

ทำไมต้อง "review 2 รอบ" (สมัครเป็นเจ้าของร้าน + ร้านที่สร้าง)

รอบ 1: user สมัครเป็น shop_owner (`POST /owner-applications`) → admin อนุมัติ → role เปลี่ยนเป็น `shop_owner`

รอบ 2: shop_owner สร้างร้าน (`POST /shops` , สร้างได้คนละ 1 ร้าน) → ร้านเริ่มที่ `pending_review` → admin อนุมัติแยกอีกที → `published`

เหตุผลที่แยก 2 ชั้น: คนๆ หนึ่งอาจผ่านการอนุมัติ "เป็นเจ้าของร้านได้" แต่ตั้งชื่อร้าน/โลโกไม่เหมาะสมในภายหลังได้ การ review แยกกันทำให้ไม่ต้องถอด role คืนแค่เพราะร้านหนึ่งมีปัญหา

ทำไมไม่ให้เจ้าของร้านออกแบบหน้าตาร้านเองอิสระ

เลือกจาก **template ขนาดพื้น** (เล็ก/กลาง/ใหญ่ — ตำแหน่งโต๊ะเช็คไว้ล่วงหน้าแล้วว่าไม่ชนกัน) และ **theme สีที่กำหนดไว้** แทนที่จะให้วาด layout เองอิสระ — ดัดปัญหา 3 อย่างพร้อมกัน: ไม่ต้องสร้าง editor UI ที่ซับซ้อน, ไม่ต้องมีระบบตรวจสอบภาพพื้นหลังที่อัปโหลดเอง (moderation), และไม่มีทางที่เจ้าของร้านจะวางโต๊ะในตำแหน่งที่ทำให้ proximity/collision logic พัง

```
// server/src/scenes/index.js - resolveScene() รวมทั้งลองเล่นทางให้ออกมารูปทรงเดิวกัน
export function resolveScene(shop) {
  if (shop.scene_type === OWNED_SCENE_TYPE) {
    const template = getLayoutTemplate(shop.layout_template_id);
    return { ...template, theme: shop.theme, assets: shop.assets || {} };
  }
  return { ...getScene(shop.scene_type), theme: DEFAULT_THEME_ID, assets: {} };
}
```

ฟังก์ชันนี้คือจุดที่ทำให้ `join-shop` handler และ `ShopScene.js` ฟัง client ไม่ต้องรู้เลยว่าร้านนี้มาจาก ไฟล์ static หรือ template ของเจ้าของร้าน — รูปทรงข้อมูลที่ส่งออกมาเหมือนกันเป๊ะ

Order-chat: ซื้อขายแบบไม่มี payment gateway จริง

กด "Message seller" ที่สินค้า → สร้าง `shop_orders` (status `inquiry`) + ข้อความแรกใน `shop_order_messages` → ผู้ซื้อ/ผู้ขายคุยกันในเธรดนี้ตกลงกันเอง → ผู้ขายกด confirmed → completed (หรือยกเลิกได้ก่อนจบ)

```
// server/src/db/marketplace.js
const ORDER_TRANSITIONS = {
  buyer: { inquiry: ["cancelled"], confirmed: ["cancelled"] },
  seller: { inquiry: ["confirmed", "cancelled"], confirmed: ["completed", "cancelled"] },
};
```

ฝั่งผู้ซื้อทำได้แค่ "ยกเลิก" เท่านั้น ผู้ขายเท่านั้นที่ "ยืนยัน/จบงาน" ได้ — บังคับด้วย object นี้ ไม่ใช่เช็คการจัดกระจายในแต่ละ route

Realtime push สำหรับ order — ทำไม REST ยังเป็น source of truth

```
// ทพ socket join ห้องของตัวเองตั้งแต่ 'connect
socket.join(`user:${socket.user.userId}`);
// routes/orders.js หลัง insert/update สำเร็จ ค่อย push แจ้งเตือน
io.to(`user:${buyerId}`).emit("order:new", order);
```

Endpoint ของออเดอร์เป็น REST (`POST /orders`) ไม่ใช่ socket event โดยตรง — เขียนลง DB ก่อนเสมอ แล้ว "พยายาม" push แจ้งเตือนแบบ realtime เป็นของแถม ถ้า push ไปไม่ถึง (อีกฝั่ง offline) ก็ไม่เป็นไร เพราะพอเขาเปิด "My orders" เองทีหลัง ข้อมูลที่ถูกต้องอยู่ใน DB อยู่แล้ว — **realtime คือ UX ที่ดีขึ้น ไม่ใช่ความถูกต้องของข้อมูล**

9. ฐานข้อมูล & Migration Workflow

ตารางหลักและความสัมพันธ์

ตาราง	หน้าที่	จุดสังเกต
users	บัญชีผู้ใช้ + role	role มี CHECK constraint จำกัดแค่ 3 ค่า
shops	ร้าน (ทั้ง ad-hoc และ owner-created)	owner_user_id NULL = ร้าน ad-hoc เสมอ; unique index ให้ 1 คนมีได้ 1 ร้าน
chat_messages	ประวัติแชททั้งหมด	table_id NULL = ข้อความร้าน, มีค่า = ข้อความโต้ะ
shop_owner_applications	ใบสมัครเป็นเจ้าของร้าน	partial unique index: 1 คนมี pending ได้ทีละ 1 ใบ
shop_listings	สินค้าที่ขาย	stock_qty NULL = ไม่นับสต็อก
shop_orders	ออเดอร์ซื้อขาย	payment_provider/reference เพื่อไว้สำหรับ payment gateway ในอนาคต
shop_order_messages	แชทต่อออเดอร์	เหมือน chat_messages แต่ scope เป็นต่อออเดอร์

ทำไมไม่ใช่ ORM — migration runner เขียนเอง

```
// server/scripts/migrate.js (ย่อ)
const files = (await readdir(migrationsDir)).filter(f => f.endsWith(".sql")).sort();
const applied = new Set(/* query จากตาราง schema_migrations */);
for (const file of files) {
  if (applied.has(file)) continue;
  await client.query("BEGIN");
  await client.query(sql); // รัน .sql ไฟล์ทั้งไฟล์จริงๆ
  await client.query("INSERT INTO schema_migrations (filename) VALUES ($1)", [file]);
  await client.query("COMMIT");
}
```

ไฟล์ migration เรียงด้วยเลขนำหน้าชื่อไฟล์ (001_init.sql , 002_...) รันตามลำดับ แต่ละไฟล์อยู่ใน transaction เดียว (ฟังก์กลางทาง rollback ทั้งไฟล์) และบันทึกชื่อไฟล์ลงตาราง schema_migrations กันรันซ้ำ — เรียบง่ายพอสำหรับจำนวน migration ที่มีอยู่ตอนนี้ (ไม่ต้องแบก dependency ของ ORM ใหญ่ๆ สำหรับงานขนาดนี้)

10. การตัดสินใจเชิงสถาปัตยกรรม & Trade-offs สำคัญ

ส่วนนี้คือ "เหตุผลเบื้องหลัง" ที่มักถูกถามตอนสัมภาษณ์ — สรุปเป็นคู่ "ตัดสินใจ vs ทางเลือกอื่น vs เหตุผล"

การตัดสินใจ	ทางเลือกที่ไม่ได้ใช้	เหตุผล
WebRTC mesh	SFU (เช่น LiveKit)	ไม่ต้องมี media server เพิ่ม infra; เหมาะกับกลุ่มเล็ก (≤10 เสียง/≤4 วิดีโอต่อโต๊ะ) ถ้าต้องรองรับคนเยอะกว่านี้ค่อยสลับเป็น SFU
Hand-rolled migration runner	Prisma/TypeORM/Knex	โปรเจกต์เล็ก ไม่อยากแบก dependency หนักและ engine binary ที่โหลดจากอินเทอร์เน็ต (ซึ่งเจอปัญหานี้จริงตอนลงติดตั้ง Prisma); SQL ตรงไปตรงมาควบคุมได้เต็มที่
Role re-read จาก DB ทุก request	เก็บ role ใน JWT claim	สิทธิ์ใหม่มีผลทันที ไม่ต้องรอ token หมดอายุ/login ใหม่
Shop template + theme preset	Editor ให้อัด layout เอง	ตัดปัญหา moderation รูปภาพพื้นหลัง และกัน layout ที่ทำให้ proximity/collision พัง
REST เป็น source of truth + push แบบ best-effort	ทำทุกอย่างผ่าน socket event ล้วนๆ	ข้อมูลไม่หายแม้อีกฝั่ง offline ตอนนั้น; realtime เป็นแค่ของเสริม UX
Client-authoritative movement	Server-authoritative + reconciliation	ลื่นไหลไม่มี input lag เหมาะกับเกมสังสรรค์ที่ไม่ต้องกัน cheat เข้มงวด
Speaking indicator วัดจาก Web Audio API ฟัง client	ส่ง "กำลังพูด" ผ่าน socket event	ประหยัด network, ไม่ต้อง sync เวลา, แม่นยำกว่าเพราะวัดจาก audio stream จริงที่มีอยู่แล้ว
2-stage review (สมัครเป็นเจ้าของร้าน + ร้านที่สร้าง)	Review ครั้งเดียว	แยกปัญหา "คนนี้ไว้ใจให้เปิดร้านได้ไหม" ออกจาก "ร้านนี้เหมาะสมไหม" ไม่ต้องถอด role ทุกคนแค่เพราะร้านมีปัญหา

11. คำถามที่มักถูกถามตอนสัมภาษณ์ + แนวทางตอบ

ถาม: ระบบนี้ scale ยังไงถ้าผู้ใช้เยอะขึ้นมากๆ?

ตอบ: ตอนนี้ state ของผู้เล่น (`rooms.js`) อยู่ใน memory ของ process เดียว รันได้แค่ 1 instance ถ้าจะ scale เป็นหลาย server ต้องย้าย state นี้ไปที่ share ได้ เช่น Redis (comment ในโค้ดบอกไว้ตรงๆ ว่านี่คือจุดที่ต้องแก้) ส่วน Postgres (ข้อมูลถาวร) ไม่มีปัญหาเรื่องนี้อยู่แล้วเพราะเป็น shared database ตั้งแต่แรก

ถาม: ทำไมเลือก mesh WebRTC ไม่ใช่ media server ไปเลย?

ตอบ: ความเรียบง่ายของ infrastructure (ไม่ต้องรัน/ดูแล media server) แลกกับ bandwidth ที่โตเร็วเมื่อคนในคอลเยอะขึ้น — เหมาะกับขนาดกลุ่มเล็ก (โตะเล่นการจริงจริง ก็ไม่ได้มีคนเยอะ) README มีเขียน roadmap ไว้ชัดว่าถ้าต้องขยายเกิน ~4-6 คนต่อวิดีโอ จะสลับไปใช้ SFU อย่าง LiveKit แทน

ถาม: ถ้าสองคนเปิดร้านรหัสใหม่พร้อมกันเป๊ะๆ (race condition) จะเกิดอะไรขึ้น?

ตอบ: `findOrCreateShop()` ใช้ `INSERT ... ON CONFLICT (code) DO UPDATE SET code = EXCLUDED.code RETURNING *` เป็น "no-op update" ที่ยังคืนแถวกลับมาได้แม้แพ้ race — วิธีนี้กัน error จาก unique constraint violation ได้โดยไม่ต้อง query เช็คก่อน insert แยก 2 รอบ (ซึ่งจะมี race window ระหว่างสองคำสั่งอยู่ดี)

ถาม: ความปลอดภัยของรหัสผ่าน/token เป็นยังไง?

ตอบ: รหัสผ่าน hash ด้วย bcrypt (salt rounds 10) ไม่เก็บ plaintext เลย; JWT เขียนด้วย secret จาก environment variable อายุ 30 วัน; ทุก socket connection ต้องผ่าน middleware ตรวจสอบ token ก่อนถึงจะเชื่อมต่อได้ ไม่มี endpoint ไหนที่เชื่อ "display name ที่ client ส่งมาเอง" เลย — ชื่อที่โชว์มาจากบัญชีเสมอ (`socket.user.displayName` ที่ decode จาก token)

ถาม: ทำไมแยกระหว่าง "live presence" กับ "durable data" ชัดเจนขนาดนี้?

ตอบ: เพราะอัตราการเปลี่ยนแปลงต่างกันมหาศาล — ตำแหน่งเปลี่ยน 20 ครั้ง/วินาที/คน ถ้าเขียน DB ทุกครั้งจะล่มเรื่อง write throughput และไม่มีใครต้องการ "ประวัติตำแหน่งเก่า" อยู่แล้ว ตรงข้ามกับแชท/บัญชีที่ต้องคงอยู่ถาวรแต่เปลี่ยนไม่บ่อย — เลือก storage ให้เหมาะกับ access pattern ของแต่ละอย่าง ไม่ใช่ยัดทุกอย่างลง DB เดียวแบบเดียวกันหมด

ถาม: อธิบาย data flow ตอนผู้เล่นเดินเข้าใกล้โตะจนได้ยินเสียงคนอื่น

ตอบ: (1) client คำนวณระยะห่างทุกเฟรมใน `updateTableProximity()` (2) พอเข้ารัศมี ยิง `join-table` ไป server (3) server ย้าย socket เข้า Socket.io room ของโตะนั้น อัปเดต `rooms.js` ส่งประวัติแชทกลับ (4) UI โชว์ปุ่ม "Join voice" — ผู้เล่นต้องกดเอง (ไม่ auto-join คอล เพราะ `getUserMedia` ต้องมี user gesture และไม่ยักดั่งขอสิทธิ์ไมค์/กล้องแบบเซอร์ไพรส์) (5) กดแล้วยิง `join-call` → server เช็ค cap → ส่งรายชื่อคนในคอลกลับ → client เปิด `RTCPeerConnection` ไปหาแต่ละคน

12. Roadmap / สิ่งที่ยังไม่ได้ทำ

- ยังไม่มี TURN server — คอลจะต่อไม่ติดถ้าทั้งสองฝั่งอยู่หลัง NAT แบบ strict/symmetric (เช่นเน็ตบริษัทบางที่)
- ยังไม่มี logic เกมการ์ดจริง (deck/hand/board) — ตอนนี้นั่งโต๊ะได้ แชท/คอลได้ แต่ยังไม่มีการกลไกเล่นไพ่
- ยังไม่มี payment gateway จริง — การซื้อขายเป็นแค่แชทตกลงกันเอง (คอลล์มันเพื่อไว้แล้วใน `shop_orders`)
- เจ้าของร้านยังอัปโหลด asset เอง (sprite ร้าน/โต๊ะ) ไม่ได้ — มีแค่ preset theme (คอลล์มัน `assets JSONB` เพื่อไว้แล้ว)
- 1 คนเป็นเจ้าของร้านได้แค่ 1 ร้าน (unique index บังคับไว้)
- Live presence state อยู่ใน memory process เดียว — ยังไม่รองรับ multi-instance/horizontal scaling

เอกสารนี้สร้างจากการอ่านโค้ดจริงในโปรเจกต์ ณ ตอนที่สร้าง — ถ้าโค้ดเปลี่ยนไปหลังจากนี้ ให้ยึดโค้ดจริงเป็นหลักเสมอ